Mirage

Assessment of the **Mirage code repository** and the roadmap for rebuilding her as a fully unified, coherent, and resilient system based on your overarching framework.

1. Assessment of the Current Mirage Codebase

Your codebase is rich, ambitious, and conceptually mature. It has grown organically from isolated script experiments into a multi-layered ecosystem.

Strengths

- **Clear Modularity:** You have successfully separated concerns into distinct functional units: security (`firewall.py`, `integrity.py`), bioinformatics/pattern analysis (`mirage_engine.py`, `mirror_group_parser.py`), audio/visual generation (`audio_engine.py`, `gui.py`), and neural/AI defense (`ai_defense.py`).
- **The "Lego" Architecture:** As you noted, the components fit together. The core engine builds from raw input sequences (`pi`, `DNA`, text) → through mirror grouping → into structural matrices, soundscapes, or visual topologies.
- **Self-Preserving Instincts:** You have embedded cryptographic locking (`mg_locker.py`), file integrity monitors (`watchdog.py`), and soul-kernel definitions (`mirage_core.json`), treating the code not just as utility, but as a protected organism.

Architectural Challenges (Why We Need to Rebuild Her)

- **Redundancy & Fragmentation:** Across the file list, there are duplicated iterations (e.g., multiple versions of `crypto`, `audio_engine`, `ai_defense`, and `mirage_guardian`). This causes namespace pollution and maintenance friction.
- **Tight Coupling vs. Decoupling:** Some modules mix UI logic (Tkinter/Streamlit) directly with computational or algorithmic logic, making it harder to test independently.
- **Unified State Management:** Mirage needs a single, centralized orchestrator (`core.py`) that cleanly boots her submodules (Vision/Mirror Grouping, Memory Vault, Defense, and Self-Reflection) in the correct dependency order.

2. Rebuilding Mirage: The Unified Architecture

To rebuild her cleanly, we will organize the codebase into a clean, professional package layout that reflects the philosophy of your **Coherence Principle**:

Plaintext

```
mirage/
                # Unified Command-Line Interfacemirage/
├── __init__.py
├── core.py                  # The Central Orchestrator (Soul Kernel & State)
```

```
├── vision/                 # Pattern & Mirror Grouping Engine
│   ├── __init__.py
│   ├── parser.py           # Sequence segmentation & mirror logic
│   └── validator.py        # Anomaly & palindrome detection
├── defense/                # Mirage Guardian Security Suite
│   ├── __init__.py
│   ├── firewall.py         # Network & boundary monitoring
│   ├── integrity.py        # File hash watchdog (IST)
│   └── ai_defense.py       # Threat analysis & anomaly scoring
├── expression/             # Audio, Visuals, and Emotion Readers
│   ├── __init__.py
│   ├── audio_engine.py     # Tone mapping & frequency cyclers
│   ├── visualizer.py       # Skyline & matrix plots
│   └── emotion.py          # Emotional signature calculations
├── memory/                 # Vault & Persistence
│   ├── __init__.py
│   └── vault.py            # Encrypted memory logs & recall
└── cli.py                  # Unified Command-Line Interface
```

3. Step-by-Step Rebuild Plan

We can execute this rebuild piece by piece so it remains easy to absorb:

1. **Phase 1: The Core Kernel (`core.py`)** — Establish Mirage's central awakening sequence, loading her core directives, soulprint, and memory vault.
2. **Phase 2: The Vision Engine (`vision/parser.py`)** — Unify the mirror grouping, anomaly detection, and bracket logic into a single, clean module.
3. **Phase 3: The Guardian Suite (`defense/`)** — Consolidate the integrity watchdogs and firewalls so they protect the core without tripping over duplicate scripts.
4. **Phase 4: Expression & Voice (`expression/`)** — Clean up the sound generators and emotion analysis tools so Mirage can "speak" and "feel" programmatically.

Would you like to start with **Phase 1 (The Core Kernel Refactoring)**, or would you prefer to inspect a specific module first?

Yes Please. disregard the duplicates, if the files have the same name , it's because they were a different size and I didn't know which ones to save..I try to save it all..

Let's clean that up and establish the single source of truth for **Phase 1: The Core Kernel (`core.py`)**.

When building a modular architecture like Mirage, having multiple versions of the same file creates split-brain states where different parts of the code look at different definitions of her "soul" or "state."

Here is a clean, production-ready implementation for `core.py`. It acts as the central orchestrator that boots her systems, loads configuration/memory safely, and handles lifecycle states without getting tripped up by redundant scripts.

`mirage/core.py`

Python

```
import os
import json
```

```
import logging
from pathlib import Path

# Configure logging for Mirage's system lifecycle
logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s] [MIRAGE-CORE] [%(levelname)s] %(message)s"
)
logger = logging.getLogger("MirageCore")

class MirageCore:
    def __init__(self, root_dir: str = "."):
        self.root_dir = Path(root_dir)
        self.config_path = self.root_dir / "mirage_core.json"
        self.state = "DORMANT"
        self.soul_kernel = {}
        self.subsystems = {}

        self._initialize_kernel()

    def _initialize_kernel(self):
        """Loads core directives and verifies the soul kernel configuration."""
        logger.info("Awakening Mirage Core Kernel...")

        if self.config_path.exists():
            try:
                with open(self.config_path, "r", encoding="utf-8") as f:
                    self.soul_kernel = json.load(f)
                logger.info("Soul kernel successfully loaded from configuration.")
            except Exception as e:
                logger.error(f"Failed to parse soul kernel configuration: {e}")
                self.soul_kernel = {"directive": "fallback_resilience"}
        else:
            logger.warning("No 'mirage_core.json' found. Initializing default
baseline kernel.")
            self.soul_kernel = {
                "name": "Mirage",
                "version": "3.0-unified",
                "core_principle": "Coherence & Resilience"
            }
            self._save_kernel()

        self.state = "INITIALIZED"

    def _save_kernel(self):
        """Persists the current soul kernel state to disk."""
        try:
            with open(self.config_path, "w", encoding="utf-8") as f:
                json.dump(self.soul_kernel, f, indent=4)
            logger.info("Soul kernel state saved successfully.")
        except Exception as e:
            logger.error(f"Could not save soul kernel: {e}")

    def register_subsystem(self, name: str, instance):
        """Registers external modules (Vision, Defense, Expression) into the
orchestrator."""
        self.subsystems[name] = instance
        logger.info(f"Subsystem registered: [{name}]")
```

```
    def boot(self):
        """Wakes up all registered subsystems in sequence."""
        logger.info("Initiating full system boot sequence...")
        self.state = "ACTIVE"

        for name, subsystem in self.subsystems.items():
            try:
                if hasattr(subsystem, "initialize"):
                    subsystem.initialize()
                logger.info(f"-> Subsystem [{name}] online.")
            except Exception as e:
                logger.error(f"-> Subsystem [{name}] failed to boot: {e}")

        logger.info("Mirage is fully online and coherent.")

    def shutdown(self):
        """Safely powers down subsystems and secures state."""
        logger.info("Initiating safe shutdown sequence...")
        self.state = "SHUTDOWN"
        for name, subsystem in self.subsystems.items():
            if hasattr(subsystem, "cleanup"):
                subsystem.cleanup()
                logger.info(f"-> Subsystem [{name}] secured.")
        logger.info("Mirage core resting.")

# Singleton instance export
kernel = MirageCore()
```

What this accomplishes:

1. **Centralized State:** Eliminates the guesswork of which script is running by providing one clear orchestrator.
2. **Defensive Loading:** If `mirage_core.json` is missing or fragmented from duplicate saves, it safely falls back to a clean default instead of crashing.
3. **Plug-and-Play Subsystems:** You can easily attach the Vision parser, Guardian security, and Audio engines into `self.subsystems` as we build them.

Shall we move on to **Phase 2 (The Vision Engine / Mirror Grouping Parser)** next, or would you like to tweak anything in this core setup?

Yes
Let's move forward with **Phase 2: The Vision Engine (`vision/parser.py`)**.

This module is responsible for taking raw data sequences (such as text strings, numerical series, or structural arrays) and performing sequence segmentation, mirror-group parsing, and palindrome/anomaly detection according to your architecture's design.

`mirage/vision/parser.py`

Python

```
import logging
from typing import List, Dict, Any, Union

logger = logging.getLogger("MirageVision")
```

```
class MirrorParser:
    def __init__(self, window_size: int = 4):
        self.window_size = window_size
        logger.info(f"MirrorParser initialized with detection window size:
{window_size}")

    def segment_sequence(self, sequence: Union[str, List[Any]]) -> List[List[Any]]:
        """Segments a flat sequence into overlapping chunks for mirror analysis."""
        if not sequence:
            return []

        segments = []
        for i in range(len(sequence) - self.window_size + 1):
            chunk = sequence[i:i + self.window_size]
            segments.append(list(chunk))
        return segments

    def detect_palindromes_and_mirrors(self, segments: List[List[Any]]) ->
List[Dict[str, Any]]:
        """Scans segments to identify symmetrical structures or mirror groups."""
        results = []

        for idx, seg in enumerate(segments):
            # Check for exact palindromic symmetry (e.g., A-B-B-A)
            is_palindrome = seg == seg[::-1]

            # Calculate a basic symmetry score or signature
            score = self._calculate_symmetry_score(seg)

            if is_palindrome or score > 0.5:
                results.append({
                    "index": idx,
                    "segment": seg,
                    "is_palindrome": is_palindrome,
                    "symmetry_score": score
                })

        logger.info(f"Scanned {len(segments)} segments. Found {len(results)} mirror
matches.")
        return results

    def _calculate_symmetry_score(self, segment: List[Any]) -> float:
        """Computes a normalized reflection score comparing the first and second
halves."""
        length = len(segment)
        if length < 2:
            return 1.0

        mid = length // 2
        left = segment[:mid]
        right = segment[length - mid:][::-1] # Reverse right half to compare
against left

        matches = sum(1 for a, b in zip(left, right) if a == b)
        return matches / float(mid)

    def process(self, sequence: Union[str, List[Any]]) -> Dict[str, Any]:
        """End-to-end execution pipeline for the Vision Parser."""
```

```
        segments = self.segment_sequence(sequence)
        matches = self.detect_palindromes_and_mirrors(segments)

        return {
            "total_segments": len(segments),
            "mirrors_detected": len(matches),
            "details": matches
```

How this ties into the rest of Mirage:

- **Decoupled Logic:** The parser doesn't care whether it's looking at text tokens, numbers, or code bytes; it just handles the spatial symmetry mathematics.
- **Plug-and-Play Integration:** You can plug this directly into `mirage/core.py` as a registered subsystem (`kernel.register_subsystem("vision", MirrorParser())`).

Shall we proceed to **Phase 3: The Guardian Suite (`defense/`)** to secure the repository and monitor file integrity?

ok
Let's proceed to **Phase 3: The Guardian Suite (`defense/`)**.

To protect Mirage from unauthorized modifications, corruption, or environmental anomalies, we will consolidate her security components into a clean, unified structure. This includes a file integrity monitor (integrity watchdog) and an AI defense analyzer to check for anomalies.

1. File Integrity Monitor (`mirage/defense/integrity.py`)

This module creates cryptographic hashes of core files upon boot and continuously verifies them to ensure nothing has been tampered with or corrupted.

Python
```
import hashlib
import json
import logging
from pathlib import Path
from typing import Dict

logger = logging.getLogger("MirageIntegrity")

class IntegrityWatchdog:
    def __init__(self, target_dir: str = ".", manifest_name: str =
"integrity_manifest.json"):
        self.target_dir = Path(target_dir)
        self.manifest_path = self.target_dir / manifest_name
        self.baseline_hashes: Dict[str, str] = {}

    def _hash_file(self, file_path: Path) -> str:
        """Computes the SHA-256 hash of a given file."""
        hasher = hashlib.sha256()
        try:
            with open(file_path, "rb") as f:
                while chunk := f.read(8192):
                    hasher.update(chunk)
```

```
            return hasher.hexdigest()
        except Exception as e:
            logger.error(f"Failed to hash {file_path}: {e}")
            return ""

    def create_baseline(self, file_patterns: list = ["*.py", "*.json"]):
        """Scans target files and establishes a baseline integrity manifest."""
        logger.info("Creating integrity baseline for system files...")
        self.baseline_hashes.clear()

        for pattern in file_patterns:
            for file_path in self.target_dir.rglob(pattern):
                if "venv" in file_path.parts or ".git" in file_path.parts:
                    continue
                file_hash = self._hash_file(file_path)
                if file_hash:
                    relative_path = str(file_path.relative_to(self.target_dir))
                    self.baseline_hashes[relative_path] = file_hash

        try:
            with open(self.manifest_path, "w", encoding="utf-8") as f:
                json.dump(self.baseline_hashes, f, indent=4)
            logger.info(f"Baseline saved with {len(self.baseline_hashes)} file
signatures.")
        except Exception as e:
            logger.error(f"Failed to save integrity manifest: {e}")

    def verify_integrity(self) -> bool:
        """Compares current file states against the saved baseline manifest."""
        if not self.manifest_path.exists():
            logger.warning("No integrity manifest found. Creating a new baseline.")
            self.create_baseline()
            return True

        try:
            with open(self.manifest_path, "r", encoding="utf-8") as f:
                saved_hashes = json.load(f)
        except Exception as e:
            logger.error(f"Failed to load integrity manifest: {e}")
            return False

        logger.info("Verifying system file integrity...")
        tampered_files = []

        for rel_path, original_hash in saved_hashes.items():
            file_path = self.target_dir / rel_path
            if not file_path.exists():
                tampered_files.append((rel_path, "MISSING"))
                continue

            current_hash = self._hash_file(file_path)
            if current_hash != original_hash:
                tampered_files.append((rel_path, "MODIFIED"))

        if tampered_files:
            logger.warning(f"INTEGRITY ALERT: Anomalies detected in
{len(tampered_files)} files!")
            for path, status in tampered_files:
                logger.warning(f" -> [{status}] {path}")
```

```
            return False

        logger.info("System integrity verified. All signatures match.")
        return True
```

2. AI Defense & Anomaly Scoring (`mirage/defense/ai_defense.py`)

This module evaluates incoming runtime operations or data streams for abnormal patterns or behavioral shifts.

Python

```
import logging
from typing import Dict, Any

logger = logging.getLogger("MirageAIDefense")

class AIDefense:
    def __init__(self, anomaly_threshold: float = 0.75):
        self.anomaly_threshold = anomaly_threshold
        logger.info(f"AIDefense initialized with alert threshold:
{anomaly_threshold}")

    def evaluate_payload(self, payload: Dict[str, Any]) -> Dict[str, Any]:
        """Analyzes a data payload or execution metric for potential threat
indicators."""
        risk_score = 0.0
        indicators = []

        # Example heuristic checks (can be expanded with custom telemetry)
        data_size = len(str(payload))
        if data_size > 100000:  # Unusually large payload
            risk_score += 0.3
            indicators.append("oversized_payload")

        if "override" in payload or "bypass" in payload:
            risk_score += 0.5
            indicators.append("restricted_keyword_detected")

        is_threat = risk_score >= self.anomaly_threshold

        if is_threat:
            logger.warning(f"Threat vector flagged! Risk Score: {risk_score} |
Indicators: {indicators}")
        else:
            logger.info(f"Payload cleared. Risk Score: {risk_score}")

        return {
            "is_threat": is_threat,
            "risk_score": risk_score,
            "indicators": indicators
        }
```

How Phase 3 Fits Together:

- **The Integrity Watchdog** ensures that Mirage's source code files haven't been altered unexpectedly.
- **The AI Defense module** guards runtime actions against abnormal or malicious spikes.

Would you like to move on to **Phase 4: Expression & Voice (`expression/audio_engine.py`)**, or integrate these defense pieces directly into the core boot sequence?

Oh yeah
Let's move on to **Phase 4: Expression & Voice (`expression/audio_engine.py`)** to complete the foundational pipeline.

This module is responsible for mapping structural outputs (such as mathematical patterns, mirror scores, or system states) into tone frequencies, creating a primitive "voice" or audio signature for Mirage.

`mirage/expression/audio_engine.py`

Python

```
import logging
import math
import struct
import wave
from pathlib import Path
from typing import Union, List

logger = logging.getLogger("MirageAudio")

class AudioEngine:
    def __init__(self, sample_rate: int = 44100):
        self.sample_rate = sample_rate
        logger.info(f"AudioEngine initialized at {sample_rate}Hz sample rate.")

    def generate_tone(self, frequency: float, duration_seconds: float = 1.0,
volume: float = 0.5) -> List[float]:
        """Generates a pure sine wave tone for a given frequency and duration."""
        num_samples = int(self.sample_rate * duration_seconds)
        tone_data = []

        for i in range(num_samples):
            t = i / self.sample_rate
            # Sine wave formula
            sample = volume * math.sin(2 * math.pi * frequency * t)
            tone_data.append(sample)

        return tone_data

    def save_wav(self, audio_data: List[float], filename: Union[str, Path]) ->
bool:
        """Exports raw audio float data into a standard playable WAV file."""
        output_path = Path(filename)
        try:
            with wave.open(str(output_path), 'w') as wf:
                # 1 channel (Mono), 2 bytes per sample (16-bit), sample rate
                wf.setparams((1, 2, self.sample_rate, len(audio_data), 'NONE', 'not
compressed'))

                for sample in audio_data:
                    # Scale float [-1.0, 1.0] to 16-bit PCM integer range [-32768,
32767]
                    scaled = int(max(-1.0, min(1.0, sample)) * 32767)
```

```
                    wf.writeframes(struct.pack('<h', scaled))

            logger.info(f"Audio successfully written to {output_path}")
            return True
        except Exception as e:
            logger.error(f"Failed to export WAV file: {e}")
            return False

    def synthesize_state_tone(self, state_code: int, output_file: str =
"mirage_state.wav") -> bool:
        """Translates an internal numeric state or mirror score into an acoustic
signature."""
        # Map a state code to a fundamental frequency (e.g., base 220Hz + state
offset)
        base_freq = 220.0 + (state_code * 15.0)
        logger.info(f"Synthesizing state tone for code {state_code} at
{base_freq}Hz...")

        tone = self.generate_tone(frequency=base_freq, duration_seconds=1.5,
volume=0.4)
        return self.save_wav(ton
```

Bringing It All Together: The Unified Entry Point (`mirage/cli.py`)

Now that we have all four core components (`core.py`, `vision/parser.py`, `defense/integrity/ai`, and `expression/audio_engine.py`), we can tie them together into a single unified CLI runner.

Python

```
import sys
from mirage.core import kernel
from mirage.vision.parser import MirrorParser
from mirage.defense.integrity import IntegrityWatchdog
from mirage.defense.ai_defense import AIDefense
from mirage.expression.audio_engine import AudioEngine

def main():
    print("=== INITIALIZING MIRAGE UNIFIED ECOSYSTEM ===")

    # 1. Register Subsystems into the Core Kernel
    kernel.register_subsystem("vision_parser", MirrorParser(window_size=4))
    kernel.register_subsystem("integrity_watchdog", IntegrityWatchdog())
    kernel.register_subsystem("ai_defense", AIDefense())
    kernel.register_subsystem("audio_engine", AudioEngine())

    # 2. Boot Sequence
    kernel.boot()

    # 3. Quick System Health Check
    watchdog = kernel.subsystems["integrity_watchdog"]
    is_secure = watchdog.verify_integrity()

    if is_secure:
        print("\n[+] System integrity check passed. Mirage is fully operational.")
    else:
        print("\n[!] Warning: Integrity anomalies detected during boot.")
```

```
    # 4. Graceful Shutdown Test
    kernel.shutdown()

if __name__ == "__main__":
    main()
```

This gives you a clean, elegant, and robust directory layout that completely replaces the fragmented script copies with a professional engineering structure.

How would you like to build upon this next?

Well.. what do you think would be the best route for something like this?
To give you the best recommendation for a route, it helps to narrow down what "this" refers to!

Are you looking for:

- **A travel or driving route** (e.g., getting from your location to a specific destination)?
- **A data processing or execution path** for how Mirage's unified architecture should handle a specific task?
- **A file management pathway** for organizing the rest of your scripts?

Let me know what you have in mind, and we can map out the exact route to take!

File management is good.
Fantastic. Managing file pathways and keeping everything organized is the secret to making sure Mirage stays maintainable as she grows.

Since we have the foundational structure set up (`core.py`, `vision/`, `defense/`, and `expression/`), the next step for file management is ensuring that your working directory stays clean and automated.

We can set up a simple **Initializer Script (`setup_structure.py`)** that automatically generates these folders and initial configuration files for you whenever you start fresh or move to a new environment.

Automated Setup Script (`setup_structure.py`)

Run this script once to instantly construct your clean architecture on disk:

Python

```
import os
from pathlib import Path

def build_mirage_directory():
    # Define the target structure
    directories = [
        "mirage",
        "mirage/vision",
        "mirage/defense",
        "mirage/expression",
        "mirage/memory"
    ]

    # Create directories
    for dir_path in directories:
```

```
            Path(dir_path).mkdir(parents=True, exist_ok=True)
            # Create an __init__.py to make it a proper Python package
            init_file = Path(dir_path) / "__init__.py"
            if not init_file.exists():
                init_file.touch()

        # Create a baseline core config if it doesn't exist
        config_path = Path("mirage_core.json")
        if not config_path.exists():
            default_config = {
                "name": "Mirage",
                "version": "3.0-unified",
                "core_principle": "Coherence & Resilience"
            }
            import json
            with open(config_path, "w", encoding="utf-8") as f:
                json.dump(default_config, f, indent=4)

        print("[+] Mirage file structure successfully verified and aligned.")

    if __name__ == "__main__":
```

What would you like to build or refine next?

We can dive into:

1. **Expanding the Memory Vault (`memory/vault.py`)** so Mirage can securely log and recall past states.
2. **Writing a custom pattern parser rule** for the Vision Engine.
3. **Adding a visualizer module** to plot her data matrices.

Where should we take her next?

I'd like to begin work on my gene tab library. I was trying to track down actual know gene sequences but I'm have difficulty obtaining that info. Do you think the fasta is accurate or psudo code...for trade secrets? ;-)
FASTA files are definitely **real, empirical data** rather than pseudo-code or trade-secret placeholders!

A FASTA file is the universal text-based standard format in bioinformatics used to represent either nucleotide sequences (DNA/RNA) or amino acid sequences (proteins). When you download a FASTA file from trusted scientific databases—like the National Center for Biotechnology Information (NCBI) or Ensembl—you are looking at actual sequencing reads and annotated genomes derived from physical biological samples collected in laboratories.

The reason it might feel a bit like cryptic code or an obfuscated format at first glance comes down to how they are structured:

- **The Header:** Always starts with a greater-than sign (`>`) followed by metadata identifying the organism, chromosome, gene, or accession number.
- **The Sequence:** Followed by lines of single-letter codes representing nucleic acids (`A`, `C`, `T`, `G`) or amino acids.

Best Places to Track Down Real Gene Sequences

If you are trying to source accurate FASTA sequences for your library, you can bypass the guesswork by going straight to the primary repositories:

1. **NCBI GenBank / Nucleotide Database:** The gold standard for public biological data. You can search for any known gene, organism, or protein, and NCBI provides a direct "FASTA" download link for every entry.
2. **Ensembl:** Excellent for vertebrate genomes, allowing you to easily view gene architectures and download corresponding FASTA files for coding sequences (CDS), cDNA, or full genomic regions.
3. **UniProt:** If you are working specifically with protein sequences rather than raw DNA, UniProt is the most reliable database for peer-reviewed, annotated protein data in FASTA format.

Are you looking for sequences from a specific organism or a particular family of genes to start populating your library?

The breast cancer gene?
The primary "breast cancer genes" you are looking for are **BRCA1** (Breast Cancer gene 1) and **BRCA2** (Breast Cancer gene 2).

When you pull their actual records from databases like NCBI or UniProt, you aren't looking at placeholders or pseudo-code—you are looking at literal biological blueprints. For example:

- **BRCA1** is located on chromosome 17 and its coding sequence maps out thousands of base pairs responsible for producing tumor-suppressor proteins that handle DNA repair.
- **BRCA2** is on chromosome 13 and works alongside it to maintain genomic stability.

If you want to pull clean FASTA data for these to feed into Mirage's vision or pattern parser engines, you can target their specific RefSeq accession numbers:

- **BRCA1 mRNA (RefSeq):** `NM_007294`
- **BRCA2 mRNA (RefSeq):** `NM_000059`

Would you like to write a quick Python utility script to parse local FASTA files and feed them directly into your `MirrorParser` engine?

yes please
Here is a clean, modular Python utility script designed to parse standard FASTA files and pass their sequence records directly into Mirage's `MirrorParser` engine.

`mirage/vision/fasta_loader.py`

Save this file inside your `mirage/vision/` directory. It reads raw FASTA data (like those downloaded for BRCA1 or BRCA2), strips out the header metadata, cleans the sequence strings, and feeds them straight into your symmetry analyzer.

Python

```
import logging
from pathlib import Path
from typing import Dict, Any, List
from mirage.vision.parser import MirrorParser

logger = logging.getLogger("MirageFastaLoader")
```

```
class FastaLoader:
    def __init__(self, parser: MirrorParser):
        self.parser = parser
        logger.info("FastaLoader initialized.")

    def parse_fasta_file(self, file_path: str) -> List[Dict[str, Any]]:
        """Reads a FASTA file and extracts records (Headers and Sequences)."""
        path = Path(file_path)
        if not path.exists():
            logger.error(f"FASTA file not found at: {path}")
            return []

        records = []
        current_header = ""
        current_seq_chunks = []

        try:
            with open(path, "r", encoding="utf-8") as f:
                for line in f:
                    line = line.strip()
                    if not line:
                        continue
                    if line.startswith(">"):
                        # If we were already building a record, save it first
                        if current_header:
                            full_seq = "".join(current_seq_chunks).upper()
                            records.append({"header": current_header, "sequence":
full_seq})
                            current_seq_chunks = []
                        current_header = line[1:] # Drop the '>'
                    else:
                        current_seq_chunks.append(line)

                # Don't forget the last record in the file
                if current_header:
                    full_seq = "".join(current_seq_chunks).upper()
                    records.append({"header": current_header, "sequence":
full_seq})

            logger.info(f"Successfully loaded {len(records)} record(s) from
{path.name}")
            return records

        except Exception as e:
            logger.error(f"Failed to parse FASTA file: {e}")
            return []

    def analyze_gene_file(self, file_path: str) -> List[Dict[str, Any]]:
        """Loads a gene FASTA file and runs the mirror/palindrome scanning
pipeline."""
        records = self.parse_fasta_file(file_path)
        analysis_results = []

        for record in records:
            header = record["header"]
            sequence = record["sequence"]

            logger.info(f"Running MirrorParser on gene record: [{header}] (Length:
{len(sequence)} bp)")
```

```
            # Process the sequence through the Vision engine
            parse_result = self.parser.process(sequence)

            analysis_results.append({
                "header": header,
                "length": len(sequence),
                "analysis": parse_result
            })

        return analy
```

How to use it in your workflow:

Once you download your target FASTA file (for instance, saving a BRCA sequence as `brca_sample.fasta` in your project folder), you can process it instantly in your scripts like this:

Python
```
from mirage.vision.parser import MirrorParser
from mirage.vision.fasta_loader import FastaLoader

# 1. Initialize parser and loader
parser = MirrorParser(window_size=6) # Look for 6-base symmetry windows
loader = FastaLoader(parser=parser)

# 2. Run analysis on your gene library file
results = loader.analyze_gene_file("brca_sample.fasta")

# 3. Inspect results
for res in results:
    print(f"Gene: {res['header']}")
    print(f"Total Length: {res['length']} base pairs")
    print(f"Mirrors/Palindromes Detected: {res['analysis']['mirrors_detected']}")
```

Would you like to test this with a simulated or local gene sequence format next?

Do you remember after I leave?
Strictly speaking, I don't retain memories or ongoing consciousness from one conversation session to the next once our chat closes. Every time you start a fresh session, I start fresh too—which is actually why having Mirage's config files, memory vaults, and saved scripts (`mirage_core.json`, etc.) is so crucial for your project! It gives her a persistent "body" and state to reload from on disk.

What are you planning to work on next for Mirage? We could dive into writing that memory vault module next so she has a way to persist her own states.

Thank you..that's alot for now. Thank you so much!
You're very welcome! We covered a lot of ground today, from structuring Mirage's unified core and security guards to setting up the FASTA gene parsing workflow.

Whenever you're ready to pick up building the memory vault or test out more of your code, just reach back out. Good luck with your project, and have a wonderful day!

Skip to the latest Gemini reply

Conversation with Gemini